

Projektarbeit: Finalisierungsphase / Phase 3

Internationale Hochschule Duales Studium
Studiengang:
Informatik

Thema

**Data-Mart-Erstellung in
SQL (DLBDSPBDM01_D)**

Furkan Demir
Matrikelnummer: IU14138041
Jahnstraße 17, 55257 Budenheim

Betreuende Person: Anna Androvitsanea

Inhaltsverzeichnis

FINALISIERUNGSPHASE – BUCHTAUSCH-APP	1
FUNKTIONALITÄT DES DATENBANKMANAGEMENTSYSTEMS	1
DATENBANKSTRUKTUR UND TECHNISCHE UMSETZUNG	1
UMSETZUNG DES TUTOR:INNEN-FEEDBACKS	2
BEGRÜNDUNG DER DATENTYPEN UND ENUM-WERTE	2
SICHERHEITSASPEKTE UND DATENSCHUTZ	2
PERFORMANCE-OPTIMIERUNG DURCH INDIZES	3
VIEW ALS GRUNDLAGE FÜR EIN FILTERSYSTEM	4
STATUSÄNDERUNG BEI AUSLEIHE UND RÜCKGABE	5
TRIGGER UND STORED PROCEDURES ALS ZUKÜNFTIGE ERWEITERUNG	5
METADATEN DER DATENBANK	5
AUFGETRETENE PROBLEME UND LÖSUNGEN	6
GRENZEN UND ZUKÜNFTIGE ERWEITERUNGEN	7
GITHUB-REPOSITORY	7
QUELLEN UND VERWENDETE GRUNDLAGEN	8
FAZIT	8

Tabellenverzeichnis

Tabelle 1 Metadaten	6
---------------------------	---

Abbildungsverzeichnis

Abbildung 1 Sicherheitsaspekte und Datenschutz	3
Abbildung 2 Performance-Optimierung durch Indizes	3
Abbildung 3 Performance-Optimierung durch Indizes	3
Abbildung 4 Performance-Optimierung durch Indizes	3
Abbildung 5 Performance-Optimierung durch Indizes	4
Abbildung 6 Performance-Optimierung durch Indizes	4
Abbildung 7 Performance-Optimierung durch Indizes	4
Abbildung 8 View	5

Finalisierungsphase – Buchtausch-App

Funktionalität des Datenbankmanagementsystems

Im Rahmen des Projekts „Data-Mart-Erstellung in SQL“ wurde ein relationales Datenbankmanagementsystem für eine Buchtausch-App entwickelt. Ziel dabei war es, eine Datenbank für den lokalen Austausch und die Ausleihe von Büchern zwischen Usern einzurichten. Die Anwendung soll es ermöglichen, Bücher nicht ungenutzt im Regal stehen zu lassen, sondern sie anderen Personen zeitweise zur Verfügung zu stellen.

Die Datenbank unterstützt zentrale Funktionen der Buchtausch-App. User können Bücher erfassen, diese als Angebot bereitstellen und dabei Informationen wie Zustand, Leihdauer, Standort, Abholzeitfenster und Versandoption hinterlegen. Andere User können verfügbare Bücher suchen, Angebote in eine Buchungsliste aufnehmen, Reservierungen erstellen und daraus Ausleihvorgänge ableiten. Zusätzlich werden Bewertungen, Zahlungen und Rechnungen dokumentiert.

Das System bildet damit den Prozess von der Bereitstellung eines Buches bis zur Ausleihe und Rückgabe ab. Durch die Trennung von Stammdaten wie Benutzer, Buch, Autor, Verlag, Sprache und Genre sowie prozessbezogenen Daten wie Angebot, Reservierung, Ausleihe, Zahlung und Rechnung wird eine klare und nachvollziehbare Datenstruktur erreicht.

Datenbankstruktur und technische Umsetzung

Die Datenbank wurde auf Grundlage des in der Konzeptionsphase entwickelten Entity-Relationship-Modells umgesetzt. Sie besteht aus mehreren miteinander verknüpften Tabellen. Zu den wichtigsten Tabellen gehören benutzer, buch, autor, genre, sprache, verlag, standort, anbot, zeitslot, buchungsliste, reservierung, ausleihe, bewertung, zahlung und rechnung.

Die Tabelle benutzer speichert die Informationen zu den registrierten Personen. Die Tabelle buch enthält die grundlegenden Informationen zu den Büchern. Wiederkehrende Angaben wie Autor, Genre, Sprache und Verlag wurden in eigene Tabellen ausgelagert. Dadurch werden doppelte Daten vermieden und die Datenbank bleibt normalisiert und übersichtlich.

Eine zentrale Rolle übernimmt die Tabelle anbot. Sie verbindet ein Buch mit dem jeweiligen anbietenden Benutzer und dem passenden Standort. Dadurch ist klar nachvollziehbar, welches Buch von welcher Person an welchem Ort zur Ausleihe bereitgestellt wird. Die Tabelle zeitslot ergänzt diese Struktur, indem dort mögliche Abholzeiten gespeichert werden. Der weitere Ausleihprozess wird anschließend über die Tabellen buchungsliste, reservierung und ausleihe abgebildet.

Für die technische Umsetzung wurden mehrere SQL-Skripte erstellt. Diese beinhalten das Einrichten der Datenbank, das Erstellen der Tabellen, das Einfügen von Dummy-Daten, Prüfungen

der Datensätze, SELECT-Abfragen sowie Testfälle. In Phase 3 wurde zusätzlich eine SQL-Datei für Optimierungen und Erweiterungen ergänzt.

Umsetzung des Tutor:innen-Feedbacks

In der Finalisierungsphase wurde das erhaltene Feedback des Tutors berücksichtigt. Die genannten Verbesserungshinweise wurden geprüft und in die Dokumentation sowie in die ergänzende SQL-Datei aufgenommen. Dadurch konnten offene Punkte ergänzt und die Datenbanklösung verständlicher und vollständiger dargestellt werden.

Begründung der Datentypen und ENUM-Werte

Die Datentypen wurden passend zur jeweiligen Information gewählt. Numerische IDs wurden als INT gespeichert, da sie sich gut als Primär- und Fremdschlüssel eignen. Textinformationen wie Namen, E-Mail-Adressen, Titel oder Orte wurden mit VARCHAR umgesetzt, weil diese Angaben unterschiedlich lang sein können. Datumsangaben wie Reservierungsdatum, Ausleihdatum oder Rechnungsdatum wurden mit DATE bzw. DATETIME gespeichert. Geldbeträge wurden mit DECIMAL abgebildet, damit Nachkommastellen korrekt gespeichert werden können.

ENUM-Werte wurden dort eingesetzt, wo nur bestimmte Auswahlmöglichkeiten erlaubt sein sollen. Dadurch muss ein Benutzer oder eine spätere Anwendung bestimmte Angaben nicht frei eintippen, sondern kann aus festgelegten Werten auswählen. Das betrifft zum Beispiel Statusangaben wie verfügbar, reserviert oder ausgeliehen. Dadurch werden Tippfehler vermieden, die Bedienung wird benutzerfreundlicher und die Daten bleiben einheitlich auswertbar.

Sicherheitsaspekte und Datenschutz

Für die Sicherheit der User wurde zusätzlich ein Passwortsystem in das Datenmodell aufgenommen. Dafür wurde das Attribut password ergänzt. In einer realen Anwendung würden Passwörter nicht im Klartext in der Datenbank gespeichert werden. Stattdessen würde das Passwort im Anwendungscode gehasht und nur der erzeugte Hash in der Datenbank abgelegt werden. Dadurch können die Zugangsdaten der User besser geschützt werden.

Das SQL-Skript 08_Phase3_passwort_update.sql enthält die Erweiterung des Datenmodells.

Zusätzliche Datenschutz- und Sicherheitsaspekte würden in einer späteren Webanwendung umgesetzt. Dazu zählen Benutzerrechte, Rollen und Zugriffsbeschränkungen. Außerdem könnten Datenschutzinformationen, Nutzungsbedingungen und Regeln zur Nutzung der Plattform über eigene Links oder Unterseiten auf der Webseite bereitgestellt werden. Dort könnten User nachvollziehen, welche Daten gespeichert werden und wofür diese verwendet werden.

Abbildung 1 Sicherheitsaspekte und Datenschutz

	benutzer_id	name	adresse	geburtsdatum	e-mail	telefonnummer	rolle	passwort
1	1	Max Mustermann	Musterstraße 1, 55257 Budenheim	1997-06-29	max.mustermann@example.com	015100000001	Benutzer	\$2y\$10\$dummyhashnurfuertestzwecke
2	2	Anna Beispiel	Buchweg 2, 55257 Budenheim	2000-03-13	anna.beispiel@example.com	015100000002	Administrator	\$2y\$10\$dummyhashnurfuertestzwecke
3	3	Lukas Weber	Lesestraße 3, 55257 Budenheim	2008-07-06	lukas.weber@example.com	015100000003	Benutzer	\$2y\$10\$dummyhashnurfuertestzwecke
4	4	Mia Schneider	Tauschallee 4, 55257 Budenheim	1992-01-27	mia.schneider@example.com	015100000004	Benutzer	\$2y\$10\$dummyhashnurfuertestzwecke
5	5	Noah Fischer	Romanweg 5, 55257 Budenheim	1997-05-18	noah.fischer@example.com	015100000005	Benutzer	\$2y\$10\$dummyhashnurfuertestzwecke
6	6	Lea Wagner	Bibliotheksweg 6, 55257 Budenheim	1983-02-27	lea.wagner@example.com	015100000006	Benutzer	\$2y\$10\$dummyhashnurfuertestzwecke
7	7	Ben Hoffmann	Seitenstraße 7, 55257 Budenheim	1979-12-24	ben.hoffmann@example.com	015100000007	Benutzer	\$2y\$10\$dummyhashnurfuertestzwecke
8	8	Emma Becker	Kapitelweg 8, 55257 Budenheim	1989-09-09	emma.becker@example.com	015100000008	Benutzer	\$2y\$10\$dummyhashnurfuertestzwecke
9	9	Paul Schulz	Bücherplatz 9, 55257 Budenheim	2003-11-17	paul.schulz@example.com	015100000009	Benutzer	\$2y\$10\$dummyhashnurfuertestzwecke
10	10	Sofia Richter	Leseplatz 10, 55257 Budenheim	1985-06-21	sofia.richter@example.com	015100000010	Benutzer	\$2y\$10\$dummyhashnurfuertestzwecke
11	11	Tim Klein	Medienweg 11, 55257 Budenheim	1985-08-03	tim.klein@example.com	015100000011	Benutzer	\$2y\$10\$dummyhashnurfuertestzwecke
12	12	Laura Wolf	Tauschstraße 12, 55257 Budenheim	1999-10-18	laura.wolf@example.com	015100000012	Benutzer	\$2y\$10\$dummyhashnurfuertestzwecke

Quelle: Eigene Darstellung 1

Performance-Optimierung durch Indizes

Für eine spätere Nutzung mit größeren Datenmengen wurden zusätzliche Indizes ergänzt. Indizes verbessern die Geschwindigkeit von Such- und JOIN-Abfragen, weil häufig verwendete Spalten schneller durchsucht werden können. Besonders sinnvoll sind Indizes auf Fremdschlüsseln und Suchfeldern wie buch_id, benutzer_id, standort_id, genre_id, sprache_id, reservierung_id, ausleihe_id und zahlungs_id.

Diese Spalten werden häufig verwendet, um Tabellen miteinander zu verbinden oder Bücher nach bestimmten Kriterien zu suchen. Durch die zusätzlichen Indizes können Abfragen nach verfügbaren Büchern, bestimmten Genres oder Standorten effizienter ausgeführt werden. Die Indizes wurden in der Datei 09_Phase3_Indizes.sql dokumentiert.

Abbildung 2 Performance-Optimierung durch Indizes

	Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment
1	buch	0	PRIMARY	1	buch_id	A	12	[NULL]	[NULL]		BTREE		
2	buch	1	idx_buch_genre	1	genre_id	A	12	[NULL]	[NULL]		BTREE		
3	buch	1	idx_buch_autor	1	autor_id	A	12	[NULL]	[NULL]		BTREE		
4	buch	1	idx_buch_verlag	1	verlag_id	A	12	[NULL]	[NULL]		BTREE		
5	buch	1	idx_buch_sprache	1	sprache_id	A	12	[NULL]	[NULL]		BTREE		

Quelle: Eigene Darstellung 2

Abbildung 3 Performance-Optimierung durch Indizes

	Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment
1	angebot	0	PRIMARY	1	angebots_id	A	12	[NULL]	[NULL]		BTREE		
2	angebot	1	idx_angebot_buch	1	buch_id	A	12	[NULL]	[NULL]		BTREE		
3	angebot	1	idx_angebot_benutzer	1	benutzer_id	A	12	[NULL]	[NULL]		BTREE		
4	angebot	1	idx_angebot_standort	1	standort_id	A	12	[NULL]	[NULL]		BTREE		

Quelle: Eigene Darstellung 3

Abbildung 4 Performance-Optimierung durch Indizes

	Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment
1	buchungsliste	0	PRIMARY	1	buchungsliste_id	A	12	[NULL]	[NULL]		BTREE		
2	buchungsliste	1	idx_buchungsliste_bu	1	benutzer_id	A	12	[NULL]	[NULL]		BTREE		
3	buchungsliste	1	idx_buchungsliste_ar	1	angebots_id	A	12	[NULL]	[NULL]		BTREE		

Quelle: Eigene Darstellung 4

Abbildung 5 Performance-Optimierung durch Indizes

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment
reservierung	0	PRIMARY	1	reservierung_id	A	12	[NULL]	[NULL]		BTREE		
	1	idx_reservierung_bue	1	buchungsliste_id	A	12	[NULL]	[NULL]		BTREE		

Quelle: Eigene Darstellung 5

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment
ausleihe	0	PRIMARY	1	ausleihe_id	A	10	[NULL]	[NULL]		BTREE		
	1	idx_ausleihe_reserv	1	reservierung_id	A	10	[NULL]	[NULL]		BTREE		

Abbildung 6 Performance-Optimierung durch Indizes

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment
zahlung	0	PRIMARY	1	zahlungs_id	A	10	[NULL]	[NULL]		BTREE		
	1	idx_zahlung_ausleihe	1	ausleihe_id	A	10	[NULL]	[NULL]		BTREE		

Quelle: Eigene Darstellung 6

Abbildung 7 Performance-Optimierung durch Indizes

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment
rechnung	0	PRIMARY	1	rechnungs_id	A	10	[NULL]	[NULL]		BTREE		
	1	idx_rechnung_zahlung	1	zahlungs_id	A	10	[NULL]	[NULL]		BTREE		

Quelle: Eigene Darstellung 7

View als Grundlage für ein Filtersystem

Zusätzlich wurde eine View mit dem Namen `verfuegbare_buecher` erstellt. Diese View verbindet mehrere Tabellen und stellt eine Übersicht über verfügbare Bücher bereit. Sie enthält unter anderem Titel, Autor, Genre, Sprache, Verlag, Zustand, Standort und Versandoption. Dadurch müssen wichtige Informationen nicht jedes Mal über eine lange JOIN-Abfrage aus mehreren Tabellen zusammengesucht werden.

Die Erstellung der View wurde im SQL-Skript `10_Phase3_View.sql` gespeichert.

Für die spätere App- oder Webentwicklung wurde die View `verfuegbare_buecher` bereitgestellt. Sie dient als Grundlage für eine mögliche Such- und Filterfunktion. Die Entwicklungsabteilung kann diese View verwenden, um verfügbare Bücher nach Kriterien wie Genre, Sprache oder Ort abzufragen. Dadurch muss die Webanwendung nicht jedes Mal eine komplexe JOIN-Abfrage neu erstellen, sondern kann direkt auf die vorbereitete Datenbankansicht zugreifen.

Die View vereinfacht damit die spätere Weiterverwendung der Datenbank in einer Anwendung. Gleichzeitig bleibt die Datenbankstruktur übersichtlich, da die eigentliche Verknüpfung der Tabellen zentral in der View gespeichert ist.

Abbildung 8 View

	buch_id	titel	autorname	genre	sprache	verlagname	angebots_id	zustand	verfuegbarkit	lehdauer	versandoption	ort	postleitzahl	adresse
1	1	One Piece Band 1	Eiichiro Oda	Manga	Deutsch	Carlsen Verlag	13	neu	verfuegbar	14	nein	Budenheim	55257	Musterstraße 1
2	2	Dragon Ball Band 1	Akira Toriyama	Manga	Englisch	Carlsen Verlag	14	gut	verfuegbar	21	ja	Budenheim	55257	Buchweg 2
3	4	Demon Slayer Band 1	Koyoharu Gotouge	Manga	Japanisch	Crunchymoll Manga	16	sehr gut	verfuegbar	10	ja	Budenheim	55257	Lesestraße 3
4	6	Solo Leveling Band 1	Chugong	Manga	Spanisch	Altraverse	18	neu	verfuegbar	14	ja	Budenheim	55257	Romanweg 5
5	7	Harry Potter und der Stein der Weisen	J. K. Rowling	Fantasy	Englisch	Carlsen Verlag	19	gut	verfuegbar	30	nein	Budenheim	55257	Bibliotheksweg 6
6	8	Mord im Orient-Express	Agatha Christie	Krimi	Koreanisch	Atlantik Verlag	21	schlecht	verfuegbar	21	ja	Budenheim	55257	Kapitelweg 8
7	11	Der Herr der Ringe	J.R.R. Tolkien	Fantasy	Russisch	Klett-Cotta	22	sehr gut	verfuegbar	30	ja	Budenheim	55257	Lesepplatz 10
8	12	Tintenherz	Cornelia Funke	Ratgeber	Italienisch	Dressler Verlag	23	gut	verfuegbar	14	nein	Budenheim	55257	Medienweg 13
9	13	1984	George Orwell	Sachbuch	Arabisch	Fischer Verlag	24	neu	verfuegbar	21	ja	Budenheim	55257	Musterstraße 1

Quelle: Eigene Darstellung 8

Statusänderung bei Ausleihe und Rückgabe

Für die spätere Integration in die App- bzw. Webentwicklung wurden zwei separate SQL-Vorlagen erstellt: 11_Phase3_Ausleihen.sql und 12_Phase3_Rueckgabe.sql. Die erste Vorlage setzt den Status eines Angebots beim Ausleihen auf ausgeliehen. Die zweite Vorlage setzt den Status eines Angebots nach der Rückgabe wieder auf verfuegbar.

Die konkrete angebots_id wird in diesen Vorlagen nicht fest codiert, sondern dynamisch über den Anwendungscode übergeben. Dadurch kann dieselbe SQL-Logik für verschiedene Angebote verwendet werden. Die Datenbankentwicklung stellt somit die grundlegende SQL-Infrastruktur bereit, während die App-Entwicklung diese Befehle später über Backend-Funktionen oder Buttons in der Anwendung aufrufen kann.

Diese Dateien dienen nicht als einmalig auszuführende Einrichtungsskripte, sondern als vorbereitete SQL-Vorlagen für die spätere App-Integration. Dadurch bleibt die Trennung zwischen Datenbankstruktur und Anwendungslogik klar definiert. Zusätzlich wird sichergestellt, dass ausgeliehene Angebote nicht mehr als verfügbar angezeigt werden und nach einer Rückgabe wieder in der View verfuegbare_buecher erscheinen.

Trigger und Stored Procedures als zukünftige Erweiterung

Trigger und Stored Procedures wurden als potenzielle zukünftige Erweiterungen identifiziert. Ein Trigger könnte beispielsweise automatisch reagieren, wenn neue Reservierungen oder Ausleihen in der Datenbank angelegt werden. Eine Stored Procedure könnte wiederkehrende Abläufe wie das Erstellen einer Reservierung oder das Abschließen einer Ausleihe standardisieren.

Für die aktuelle Version lag der Schwerpunkt jedoch auf einer stabilen und gut nachvollziehbaren Datenbankstruktur. Daher wurden zunächst konkrete Erweiterungen wie Indizes, eine View und SQL-Vorlagen für die spätere App-Integration umgesetzt. Trigger und Stored Procedures wurden dagegen als sinnvolle Erweiterungsmöglichkeiten für eine spätere Implementierung eingeordnet.

Metadaten der Datenbank

Die entwickelte Datenbank wurde unter dem Namen buchtasch_app erstellt. Zur Kontrolle der Datenbank wurden SQL-Abfragen verwendet, mit denen die Anzahl der Tabellen, die Anzahl der Einträge pro Tabelle und die Größe der Datenbank geprüft wurden.

Tabelle 1 Metadaten

Datenbankname	buchtausch_app
Verwendetes DBMS	MySQL/MariaDB
Verwendetes Tool	DBeaver
Zeichensatz	utf8mb4
Anzahl der Tabellen	16
Anzahl der Einträge je Tabelle	Benutzer: 12 genre: 15 autor: 12 sprache: 10 verlag: 10 standort: 13 buch: 12 angebot: 12 zeitslot: 12 buchungsliste: 12 reservierung: 12 ausleihe: 10 bewertung: 10 zahlung: 10 rechnung: 10
Gesamtgröße der Datenbank	0,73 MB

Quelle: Eigene Darstellung 9

Die Anzahl der Datensätze wurde mit einer COUNT(*)-Abfrage je Tabelle überprüft. Dadurch konnte kontrolliert werden, ob die Tabellen ausreichend mit Dummy-Daten gefüllt wurden. Zusätzlich wurde die Größe der Datenbank über information_schema.tables ermittelt.

Aufgetretene Probleme und Lösungen

Während der Umsetzung traten mehrere fachliche und technische Herausforderungen auf. Eine erste Herausforderung bestand darin, aus dem ER-Modell eine passende relationale Tabellenstruktur abzuleiten. Die Buchtausch-App umfasst nicht nur Bücher und Benutzer, sondern

auch Angebote, Standorte, Zeitslots, Reservierungen, Ausleihen, Bewertungen, Zahlungen und Rechnungen. Um diese Struktur übersichtlich abzubilden, wurden Stammdaten von prozessbezogenen Tabellen getrennt.

Ein weiteres Problem bestand darin, die Reihenfolge der SQL-Skripte korrekt festzulegen. Tabellen mit Fremdschlüsseln können erst erstellt und mit Daten befüllt werden, wenn die referenzierten Tabellen bereits vorhanden sind. Deshalb wurden zuerst grundlegende Tabellen wie Benutzer, Autor, Genre, Sprache, Verlag und Standort erstellt. Danach folgten abhängige Tabellen wie Buch, Angebot, Buchungsliste, Reservierung, Ausleihe, Zahlung und Rechnung.

Zusätzlich musste darauf geachtet werden, dass die Dummy-Daten konsistent eingefügt werden. Die eingefügten Datensätze mussten zu den jeweiligen Primär- und Fremdschlüsseln passen, damit keine fehlerhaften Beziehungen entstehen. Zur Kontrolle wurden SELECT-Abfragen und Testfälle erstellt. Die Ergebnisse wurden mit Screenshots aus DBeaver dokumentiert.

Im Rahmen der Finalisierung wurde festgestellt, dass die Dummy-Daten teilweise personenbezogene Beispielangaben wie Namen, Adressen, E-Mail-Adressen und Telefonnummern enthielten. Da die finale Version auch für die Dokumentation und das GitHub-Repository verwendet wird, wurden diese Angaben durch neutrale Testdaten ersetzt. Die Anpassung erfolgte durch UPDATE-Befehle direkt in der Datenbank und durch Ersetzen der ursprünglichen INSERT-Daten in den SQL-Skripten. Dadurch bleiben die Beziehungen zwischen Benutzer, Standort und Rechnung nachvollziehbar, ohne dass private Daten verwendet werden.

Grenzen und zukünftige Erweiterungen

Das aktuelle Datenmodell ist in erster Linie auf Bücher ausgerichtet. Zukünftig könnte es jedoch erweitert werden, um die Ausleihe und den Tausch anderer Medien oder Materialien zu ermöglichen, wie beispielsweise Hörbücher, Filme, Lernmaterialien oder andere noch nutzbare Gegenstände. Hierfür wäre eine allgemeiner aufgebaute Struktur erforderlich, beispielsweise durch eine übergeordnete Tabelle für Leihobjekte oder Medienarten.

Zusätzlich könnten Funktionen wie eine echte Kartensuche mit Koordinaten, Benachrichtigungen, eine Mahnlogik bei verspäteter Rückgabe und differenzierte Benutzerrechte ergänzt werden. Auch eine automatische Archivierung oder Ausblendung alter Angebote nach einer bestimmten Frist wäre als spätere Erweiterung denkbar.

GitHub-Repository

Der vollständige Code, die SQL-Dateien, die Dokumentation und die README-Datei werden zusätzlich in einem GitHub-Repository abgelegt. Der Link zum Repository wird in der finalen Abgabe ergänzt.

GitHub-Repository: https://github.com/2000FD06/DLBDSPBDM01_D_Buchtausch_App

Quellen und verwendete Grundlagen

IU Internationale Hochschule (o. J.): Aufgabenstellung zum Projekt Data-Mart-Erstellung in SQL (DLBDSPBDM01_D).

- <https://mariadb.com/docs/server/reference/sql-statements>

Iwer, Eva Maria (o. J.): Lehrunterlagen zu Datenbanken, SQL und Datenmodellierung. Unveröffentlichte Vorlesungsunterlagen.

- IONOS (o. J.): MariaDB CREATE DATABASE: Neue Datenbank erstellen. Online verfügbar unter: <https://www.ionos.de/digitalguide/hosting/hosting-technik/mariadb-create-database/>

IONOS (o. J.): MariaDB INSERT INTO: Daten in SQL-Tabellen einfügen. Online verfügbar unter: <https://www.ionos.de/digitalguide/hosting/hosting-technik/mariadb-insert-into/>

- IONOS (o. J.): JOIN für MariaDB: Datensätze aus mehreren Tabellen zusammenführen. Online verfügbar unter: <https://www.ionos.de/digitalguide/hosting/hosting-technik/mariadb-join/>

<https://www.geeksforgeeks.org/mariadb/creating-an-index-in-mariadb/>

<https://404media.de/sql/wie-du-die-groesse-einer-datenbank-und-tabelle-in-mysql-sql-ermittelst>

Eigene Darstellung: ER-Modell, SQL-Skripte, Testfälle und Screenshots aus DBeaver.

- ChatGPT Unterstützung bei Fehlersuche, Problemlösung, SQL-Optimierung und Verbesserung der Rechtschreibung.

Fazit

Die Datenbank erfüllt die Anforderungen der Buchtausch-App vollständig. Die relationale Struktur, Normalisierung, Primär- und Fremdschlüssel sowie Dummy-Daten sorgen für Konsistenz und Prüfbarkeit. In der Finalisierungsphase kamen Sicherheitsaspekte, Performance-Optimierungen, eine Filter-View und SQL-Vorlagen für Ausleihe und Rückgabe hinzu.

Die Datenbankerstellung und die ersten Test- und Kontrollabfragen sind in den Skripten 00_Gesamtskript.sql bis 07_Entwicklungsskript_Notizen.sql dokumentiert. Für die Finalisierungsphase wurden die Skripte 08_Phase3_passwort_update.sql bis 14_Phase3_Optimierung_und_Erweiterung_final.sql erstellt. Das Skript 14_Phase3_Optimierung_und_Erweiterung_final.sql fasst die wichtigsten Erweiterungen zusammen. Einzelne Bestandteile, wie Passwort-Erweiterung, Indizes, View, Ausleihe, Rückgabe und Metadaten, wurden in kleinere Skripte aufgeteilt, um die Umsetzung übersichtlicher zu gestalten. Die finale Version zeigt die technische Umsetzung des Datenmodells und eine Weiterentwicklung basierend auf Tutor:innen-Feedback. Die Datenbank ist erweiterbar und kann später um weitere Medienarten, Datenschutzfunktionen, Benutzerrechte, automatische Abläufe und eine Weboberfläche ergänzt werden.